

Week 2: Product and Category Management System

-YAMINI S

Scenario

The e-commerce company wants to maintain information about products and their brands.

Each product belongs to a specific brand, and the system should manage product details such as product name, price, and stock quantity.

1. Design Product and Category Tables

A stores information about the different categories used to classify products.

```
SQL> CREATE TABLE Category(Category_ID VARCHAR2(7),Category_Name VARCHAR2(20),Description VARCHAR(50));  
Table created.
```

2. Product Table

Definition

A Product Table stores information about the products available in the e-commerce system, such as product name, price, stock, and category. The PDF's scenario specifically mentions managing product details such as name, price, and stock availability.

```
SQL> CREATE TABLE Product(Product_ID VARCHAR2(8),Product_Name VARCHAR(30),Category_ID VARCHAR(8),Price NUMBER(9,2),Stock_Quantity Number);  
Table created.
```

2. Define Relationships and Constraints

Relationship

One brand can have many products.

Primary Key

A Primary Key uniquely identifies each row in a table.

- Brand_ID → Primary Key in Brand table
- Product_ID → Primary Key in Product table

```
SQL> ALTER TABLE Category  
2 ADD CONSTRAINT PK_Category PRIMARY KEY(Category_ID);  
Table altered.  
  
SQL> ALTER TABLE Product  
2 ADD CONSTRAINT PK_Product PRIMARY KEY (product_ID);  
Table altered.
```

Foreign Key

A Foreign Key is used to connect two tables.

Here, Brand_ID in the Product table acts as the Foreign Key.

```
SQL> ALTER TABLE Product
  2  ADD CONSTRAINT FK_Product_Category
  3  FOREIGN KEY(Category_ID) REFERENCES Category(Category_ID);

Table altered.
```

NOT NULL

NOT NULL ensures that a column cannot contain an empty value.

For example, every brand and product should have a name.

```
SQL> ALTER TABLE Category MODIFY Category_Name VARCHAR(100) NOT NULL;

Table altered.

SQL> ALTER TABLE Product MODIFY Product_Name VARCHAR(100) NOT NULL;

Table altered.
```

CHECK

CHECK validates whether a value satisfies a condition.

Here, the product price must be greater than zero.

```
SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Price CHECK (PRICE > 0);

Table altered.

SQL> ALTER TABLE Product ADD CONSTRAINT CHK_Stock CHECK (Stock_Quantity >= 0);

Table altered.
```

UNIQUE

UNIQUE prevents duplicate values.

For example, two brands should not have the same brand name.

```
SQL> ALTER TABLE Category ADD CONSTRAINT UQ_Category_Name UNIQUE (Category_Name);

Table altered.
```

1. Insert Values

Definition

Insert Values means adding new records or data into a table.

In this system, values are inserted into the Category Table and Product Table.

```
SQL> INSERT INTO Category VALUES ('CAT101', 'Electronics', 'Electronics Gadgets and Devices');
1 row created.
```

```
SQL> SELECT * FROM Category;
```

```
CATEGOR
```

```
-----  
CATEGORY_NAME
```

```
-----  
DESCRIPTION
```

```
-----  
CAT101  
Electronics  
Electronics Gadgets and Devices
```

```
SQL> INSERT INTO Category VALUES ('CAT102', 'Jewelry', 'Handmade Jewelry and Accessories');
1 row created.
```

```
SQL> SELECT * FROM Category;
```

```
CATEGOR
```

```
-----  
CATEGORY_NAME
```

```
-----  
DESCRIPTION
```

```
-----  
CAT101  
Electronics  
Electronics Gadgets and Devices
```

```
CAT102  
Jewelry  
Handmade Jewelry and Accessories
```

```
CATEGOR
```

```
-----  
CATEGORY_NAME
```

```
-----  
DESCRIPTION
```

PRODUCT TABLE:

```
SQL> INSERT INTO Product VALUES ('P102', 'Bluetooth Speaker', 'CAT101', 1800, 15)
;

1 row created.

SQL> INSERT INTO Product VALUES ('P103', 'Resin Keychain', 'CAT104', 250, 20);

1 row created.

SQL> INSERT INTO Product VALUES ('P104', 'Resin Earrings', 'CAT102', 500, 12);

1 row created.

SQL> INSERT INTO Product VALUES ('P105', 'Wall Decor', 'CAT103', 1200, 8);

1 row created.

SQL> SELECT * FROM Product;
```

```
PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE  STOCK_QUANTITY
-----
P101
Smart Watch
CAT101          2500           10
```

```
P102
Bluetooth Speaker
CAT101          1800           15
```

```
PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE  STOCK_QUANTITY
-----

P103
Resin Keychain
CAT104          250            20
```

```
P104
Resin Earrings
```

```
PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE  STOCK_QUANTITY
-----
CAT102          500            12

P105
Wall Decor
CAT103          1200            8
```

CRUD Operations

CRUD means:

Operation SQL Command

Create INSERT

Read SELECT

Update UPDATE

Delete DELETE

A. Insert Record — CREATE

```
SQL> INSERT INTO Product VALUES ('P106', 'Resin Pendant', 'CAT102', 750, 10);
1 row created.
```

B. READ RECORD

```
SQL> INSERT INTO Product VALUES ('P106', 'Resin Pendant', 'CAT102', 750, 10);
1 row created.
SQL> SELECT * FROM Product;

PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE STOCK_QUANTITY
-----
P101
Smart Watch
CAT101         2500         10

P102
Bluetooth Speaker
CAT101         1800         15

PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE STOCK_QUANTITY
-----
P103
Resin Keychain
CAT104         250          20

P104
Resin Earrings

PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE STOCK_QUANTITY
-----
CAT102         500          12

P105
Wall Decor
CAT103        1200          8

P106

PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE STOCK_QUANTITY
-----
Resin Pendant
CAT102         750          10

6 rows selected.
```

C. Update Price

```
SQL> UPDATE Product SET Price = 300 WHERE Product_ID = 'P103';
```

```
1 row updated.
```

```
SQL> SELECT * FROM Product;
```

```
PRODUCT_
```

```
-----  
PRODUCT_NAME
```

```
-----  
CATEGORY      PRICE STOCK_QUANTITY
```

```
-----  
P101
```

```
Smart Watch
```

```
CAT101
```

```
2500
```

```
10
```

```
P102
```

```
Bluetooth Speaker
```

```
CAT101
```

```
1800
```

```
15
```

```
PRODUCT_
```

```
-----  
PRODUCT_NAME
```

```
-----  
CATEGORY      PRICE STOCK_QUANTITY
```

```
-----  
P103
```

```
Resin Keychain
```

```
CAT104
```

```
300
```

```
20
```

```
P104
```

```
Resin Earrings
```

```
PRODUCT_
```

```
-----  
PRODUCT_NAME
```

```
-----  
CATEGORY      PRICE STOCK_QUANTITY
```

```
-----  
CAT102
```

```
500
```

```
12
```

```
P105
```

```
Wall Decor
```

```
CAT103
```

```
1200
```

```
8
```

```
P106
```

```
PRODUCT_
```

```
-----  
PRODUCT_NAME
```

```
-----  
CATEGORY      PRICE STOCK_QUANTITY
```

```
Resin Pendant
```

```
CAT102
```

```
750
```

```
10
```

```
6 rows selected.
```

Delete a Product

```
SQL> DELETE FROM Product WHERE Product_ID = 'P106';
1 row deleted.
SQL> SELECT * FROM Product;
PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE STOCK_QUANTITY
-----
P101
Smart Watch
CAT101         2500         10
P102
Bluetooth Speaker
CAT101         1800         15
PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE STOCK_QUANTITY
-----
P103
Resin Keychain
CAT104         300          20
P104
Resin Earrings
PRODUCT_
-----
PRODUCT_NAME
-----
CATEGORY      PRICE STOCK_QUANTITY
-----
CAT102         500          12
P105
Wall Decor
CAT103        1200           8
```

Category-wise Product Reports

The document asks for five reports: display products under each category, count products, highest-priced product, categories with more than 5 products, and average price category-wise.

1. Display all products under each category

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price, p.Stock_Quantity
2 FROM Category c
3 JOIN Product p
4 ON c.Category_ID = p.Category_ID;
```

```
CATEGORY_NAME
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Electronics
Smart Watch
2500 10
Electronics
Bluetooth Speaker
1800 15
CATEGORY_NAME
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
Resin Art
Resin Keychain
300 20
Jewelry
Resin Earrings
CATEGORY_NAME
-----
PRODUCT_NAME
-----
PRICE STOCK_QUANTITY
-----
500 12
Home Decor
Wall Decor
1200 8
```

2. Count number of products in each category

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
2 FROM Category c
3 LEFT JOIN Product p
4 ON c.Category_ID = p.Category_ID
5 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
-----
PRODUCT_COUNT
-----
Resin Art
1
Jewelry
1
Electronics
2
CATEGORY_NAME
-----
PRODUCT_COUNT
-----
Home Decor
1
```

3. Find highest-priced product in each category

```
SQL> SELECT c.Category_Name, MAX(p.Price) AS Highest_Price
2 FROM Category c
3 JOIN Product p
4 ON c.Category_ID = p.Category_ID
5 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
-----
HIGHEST_PRICE
-----
Resin Art
      300

Jewelry
      500

Electronics
     2500

CATEGORY_NAME
-----
HIGHEST_PRICE
-----
Home Decor
     1200
```

4. Display categories having more than 5 products

```
SQL> SELECT c.Category_Name, COUNT(p.Product_ID) AS Product_Count
2 FROM Category c
3 JOIN Product p
4 ON c.Category_ID = p.Category_ID
5 GROUP BY c.Category_Name
6 HAVING COUNT(p.Product_ID) > 5;

no rows selected
```

5. Find average product price category-wise

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Average_Price
2 FROM Category c
3 JOIN Product p
4 ON c.Category_ID = p.Category_ID
5 GROUP BY c.Category_Name;
```

```
CATEGORY_NAME
-----
AVERAGE_PRICE
-----
Resin Art
      300

Jewelry
      500

Electronics
     2150

CATEGORY_NAME
-----
AVERAGE_PRICE
-----
Home Decor
     1200
```